Abstract Stack

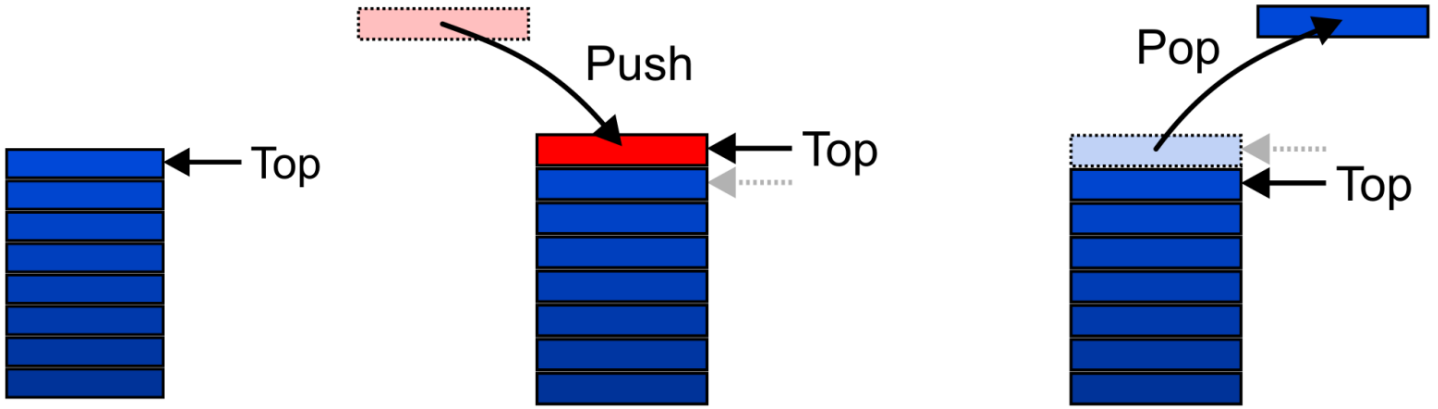
An Abstract Stack (Stack ADT) is an abstract data type which emphasizes specific operations:

- Uses a explicit linear ordering
- Insertions and removals are performed individually
- Inserted objects are *pushed onto* the stack
- The *top* of the stack is the most recently object pushed onto the stack
- When an object is *popped* from the stack, the current *top* is erased

Abstract Stack

Also called a *last-in–first-out* (LIFO) behaviour

– Graphically, we may view these operations as follows:



There are two exceptions associated with abstract stacks:

– It is an undefined operation to call either pop or top on an empty stack

Applications

Numerous applications:

- Parsing code:
  - Matching parenthesis
  - XML (e.g., XHTML)
- Tracking function calls
- Dealing with undo/redo operations
- Reverse-Polish calculators
- Assembly language

The stack is a very simple data structure

- Given any problem, if it is possible to use a stack, this significantly simplifies the solution

Stack: Applications

Problem solving:

- Solving one problem may lead to subsequent problems
- These problems may result in further problems
- As problems are solved, your focus shifts back to the problem which lead to the solved problem

Notice that function calls behave similarly:

- A function is a collection of code which solves a problem

Reference: Donald Knuth

Implementations

We will look at two implementations of stacks:

The optimal asymptotic run time of any algorithm is $\Theta(1)$

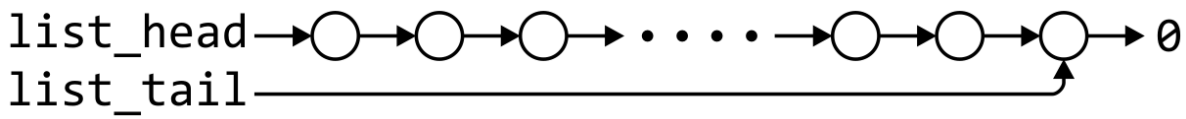
- The run time of the algorithm is independent of the number of objects being stored in the container
- We will always attempt to achieve this lower bound

We will look at

- Singly linked lists
- One-ended arrays

Linked-List Implementation

Operations at the front of a singly linked list are all $\Theta(1)$



	Front/1^{st}	Back/n^{th}
Find	$\Theta(1)$	$\Theta(1)$
Insert	$\Theta(1)$	$\Theta(1)$
Erase	$\Theta(1)$	$\Theta(n)$

The desired behaviour of an Abstract Stack may be reproduced by performing all operations at the front

Single_list Definition

The definition of single list class from Project 1 is:

```
template <typename Type>
class Single_list {
          public:
                    Single_list();
                    ~Single_list();
                    int size() const;
                    bool empty() const;
                    Type front() const;
                    Type back() const;
                    Single_node<Type> *head() const;
                    Single_node<Type> *tail() const;
                    int count( const Type & ) const;

                    void push_front( const Type & );
                    void push_back( const Type & );
                    Type pop_front();
                    int erase( const Type & );
};
```

Stack-as-List Class

The stack class using a singly linked list has a single private member variable:

```
template <typename Type>
class Stack {
    private:
        Single_list<Type> list;
    public:
        bool empty() const;
        Type top() const;
        void push( const Type & );
        Type pop();
};
```

Stack-as-List Class

A constructor and destructor is not needed

- Because `list` is declared, the compiler will call the constructor of the `Single_list` class when the `Stack` is constructed

```
template <typename Type>
class Stack {
    private:
        Single_list<Type> list;
    public:
        bool empty() const;
        Type top() const;
        void push( const Type & );
        Type pop();
};
```

Stack-as-List Class

The empty and push functions just call the appropriate functions of the `Single_list` class

```
template <typename Type>
bool Stack<Type>::empty() const {
    return list.empty();
}

template <typename Type>
void Stack<Type>::push( const Type &obj ) {
    list.push_front( obj );
}
```

Stack-as-List Class

The top and pop functions, however, must check the boundary case:

```
template <typename Type>
Type Stack<Type>::top() const {
    if ( empty() ) {
        throw underflow();
    }

    return list.front();
}
```

```
template <typename Type>
Type Stack<Type>::pop() {
    if ( empty() ) {
        throw underflow();
    }

    return list.pop_front();
}
```

Array Implementation

For one-ended arrays, all operations at the back are $\Theta(1)$

	Front/1^{st}	Back/n^{th}
Find	$\Theta(1)$	$\Theta(1)$
Insert	$\Theta(n)$	$\Theta(1)$
Erase	$\Theta(n)$	$\Theta(1)$

Destructor

We need to store an array:

- In C++, this is done by storing the address of the first entry

```
Type *array;
```

We need additional information, including:

- The number of objects currently in the stack

```
int stack_size;
```

- The capacity of the array

```
int array_capacity;
```

Stack-as-Array Class

We need to store an array:

- In C++, this is done by storing the address of the first entry

```
template <typename Type>
class Stack {
    private:
        int stack_size;
        int array_capacity;
        Type *array;
    public:
        Stack( int = 10 );
        ~Stack();
        bool empty() const;
        Type top() const;
        void push( const Type & );
        Type pop();
};
```

Constructor

The class is only storing the address of the array

- We must allocate memory for the array and initialize the member variables
- The call to `new Type[array_capacity]` makes a request to the operating system for `array_capacity` objects

```
template <typename Type>
Stack<Type>::Stack( int n ):
stack_size( 0 ),
array_capacity( std::max( 1, n ) ),
array( new Type[array_capacity] )
{
    // Empty constructor
}
```

Constructor

Warning: in C++, the variables are initialized in the order in which they are defined:

```
template <typename Type>
Stack<Type>::Stack( int n ):
stack_size( 0 ),
array_capacity( std::max( 1, n ) ),
array( new Type[array_capacity] )
{
    // Empty constructor
}
```

```
template <typename Type>
class Stack {
    private:
        int stack_size;
        int array_capacity;
        Type *array;
    public:
        Stack( int = 10 );
        ~Stack();
        bool empty() const;
        Type top() const;
        void push( const Type & );
        Type pop();
};
```

Destructor

The call to new in the constructor requested memory from the operating system

- The destructor must return that memory to the operating system:

```
template <typename Type>
Stack<Type>::~Stack() {
    delete [] array;
}
```

Empty

The stack is empty if the stack size is zero:

```
template <typename Type>
bool Stack<Type>::empty() const {
    return ( stack_size == 0 );
}
```

The following is unnecessarily tedious:

- The == operator evaluates to either `true` or `false`

```
if ( stack_size == 0 ) {
    return true;
} else {
    return false;
}
```

Top

If there are n objects in the stack, the last is located at index $n - 1$

```
template <typename Type>
Type Stack<Type>::top() const {
    if ( empty() ) {
        throw underflow();
    }

    return array[stack_size - 1];
}
```

Pop

Removing an object simply involves reducing the size

- It is invalid to assign the last entry to “0”
- By decreasing the size, the previous top of the stack is now at the location `stack_size`

```
template <typename Type>
Type Stack<Type>::pop() {
    if ( empty() ) {
        throw underflow();
    }

    --stack_size;
    return array[stack_size];
}
```

Push

Pushing an object onto the stack can only be performed if the array is not full

```
template <typename Type>
void Stack<Type>::push( const Type &obj ) {
    if ( stack_size == array_capacity ) {
        throw overflow();  // Best solution?????
    }

    array[stack_size] = obj;
    ++stack_size;
}
```

Reverse-Polish Notation

Normally, mathematics is written using what we call *in-fix* notation:

$$(3 + 4) \times 5 - 6$$

The operator is placed between to operands

One weakness: parentheses are required

$$(3 + 4) \times 5 - 6 = 29$$

$$3 + 4 \times 5 - 6 = 17$$

$$3 + 4 \times (5 - 6) = -1$$

$$(3 + 4) \times (5 - 6) = -7$$

Reverse-Polish Notation

Alternatively, we can place the operands first, followed by the operator:

$$(3 + 4) \times 5 - 6$$

$$3\ 4\ +\ 5\ \times\ 6\ -$$

Parsing reads left-to-right and performs any operation on the last two operands:

$$3\ 4\ +\ 5\ \times\ 6\ -$$

$$7\quad 5\ \times\ 6\ -$$

$$35\quad 6\ -$$

$$29$$

Reverse-Polish Notation

Other examples:

$$
\begin{array}{cccccccc}
3 & 4 & 5 & \times & + & 6 & - \\
3 & & 20 & & + & 6 & - \\
 & & 23 & & & 6 & - \\
 & & & & & 17 &
\end{array}
$$

$$
\begin{array}{cccccccc}
3 & 4 & 5 & 6 & - & \times & + \\
3 & 4 & & -1 & & \times & + \\
3 & & & -4 & & & + \\
 & & & -1 & &
\end{array}
$$

Reverse-Polish Notation

Benefits:

- no ambiguity and no brackets are required
- this is the same process used by a computer to perform computations:
  - operands must be loaded into registers before operations can be performed on them
- reverse-Polish can be processed using stacks

Reverse-Polish Notation

Reverse-Polish notation is used with some programming languages

- *e.g.*, postscript, pdf, and HP calculators

Similar to the thought process required for writing assembly language code

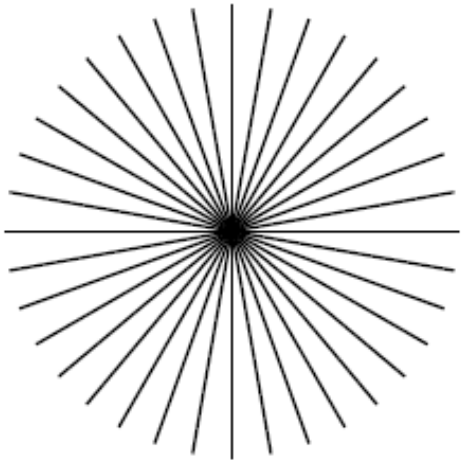
- you cannot perform an operation until you have all of the operands loaded into registers

```
MOVE.L #$2A, D1       ; Load  42 into Register D1
MOVE.L #$100, D2      ; Load 256 into Register D2
ADD D2, D1            ; Add D2 into D1
```

Reverse-Polish Notation

A quick example of postscript:

```
0 10 360 {                  % Go from 0 to 360 degrees in 10-degree steps
    newpath                 % Start a new path
    gsave                   % Keep rotations temporary
        144 144 moveto
        rotate              % Rotate by degrees on stack from 'for'
        72 0 rlineto
        stroke
    grestore                % Get back the unrotated state
} for % Iterate over angles
```



http://www.tailrecursive.org/postscript/examples/rotate.html

Reverse-Polish Notation

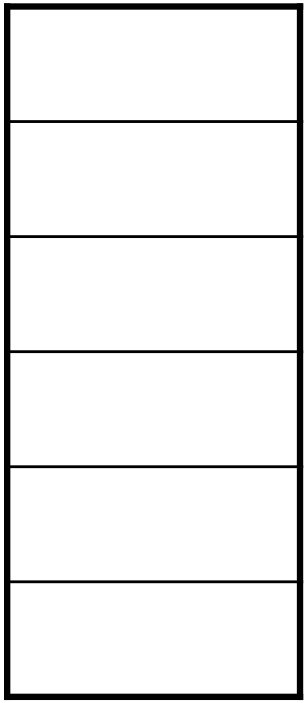
The easiest way to parse reverse-Polish notation is to use an operand stack:

- operands are processed by pushing them onto the stack
- when processing an operator:
  - pop the last two items off the operand stack,
  - perform the operation, and
  - push the result back onto the stack

Reverse-Polish Notation

Evaluate the following reverse-Polish expression using a stack:

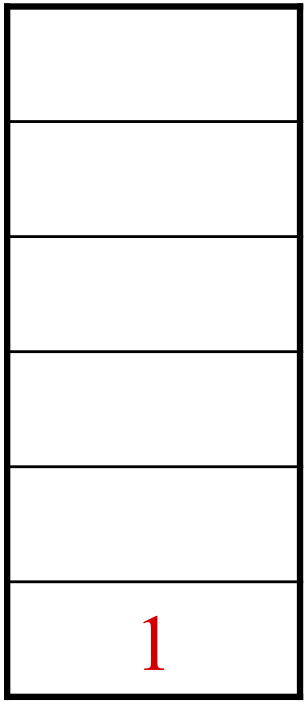
$$1\ 2\ 3\ +\ 4\ 5\ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ 9\ \times\ +$$



Reverse-Polish Notation

Push 1 onto the stack

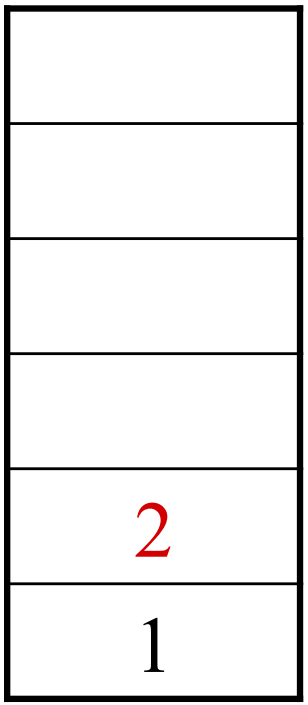
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Push 1 onto the stack

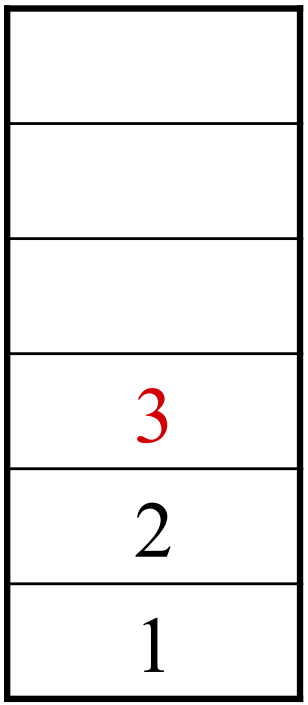
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Push 3 onto the stack

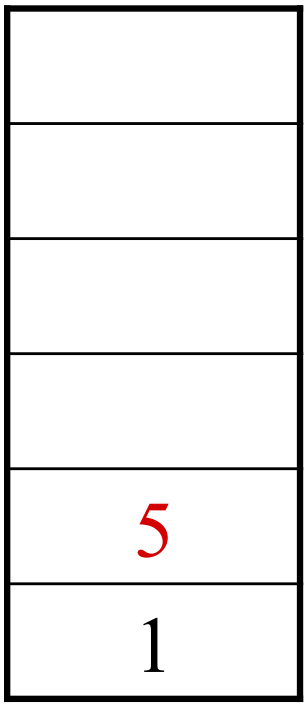
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Pop 3 and 2 and push 2 + 3 = 5

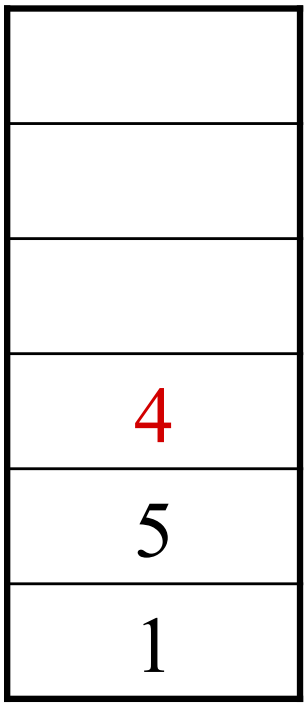
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Push 4 onto the stack

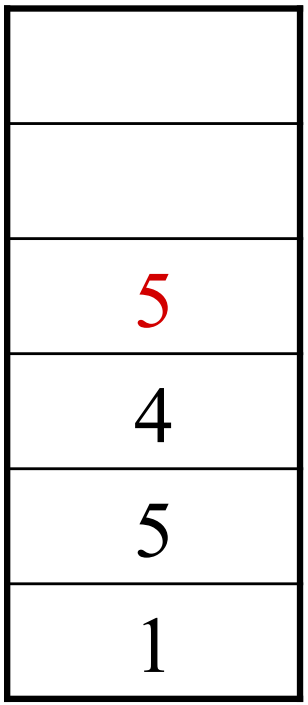
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Push 5 onto the stack

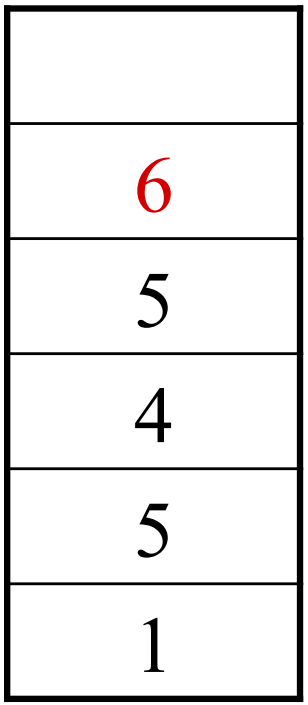
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Push 6 onto the stack

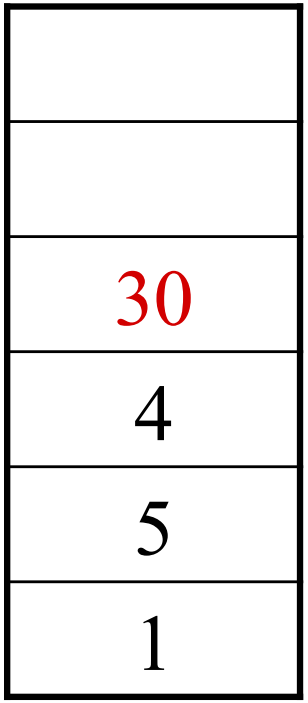
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Pop 6 and 5 and push $5 \times 6 = 30$

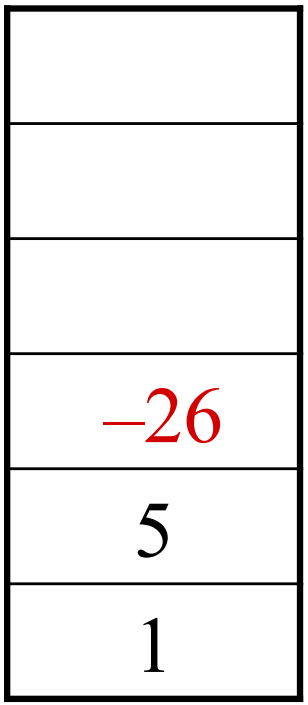
$$1\ 2\ 3\ +\ 4\ 5\ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ 9\ \times\ +$$



Reverse-Polish Notation

Pop 30 and 4 and push 4 – 30 = –26

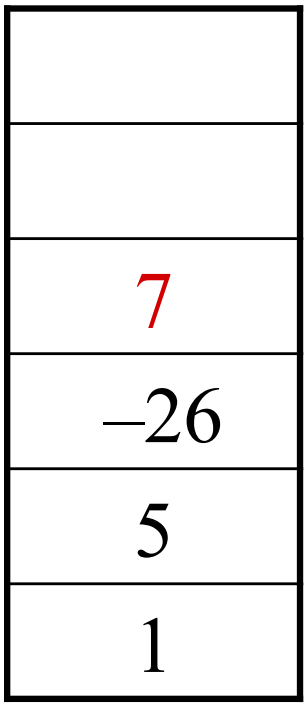
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Push 7 onto the stack

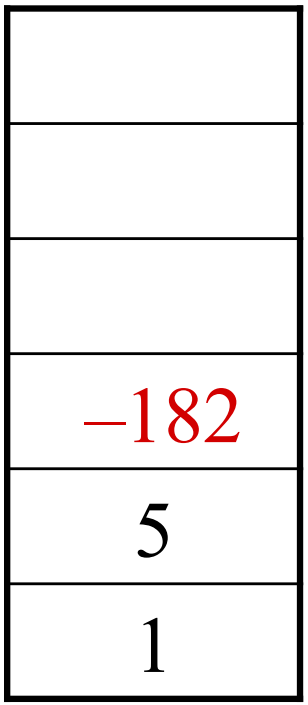
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Pop 7 and –26 and push –26 × 7 = –182

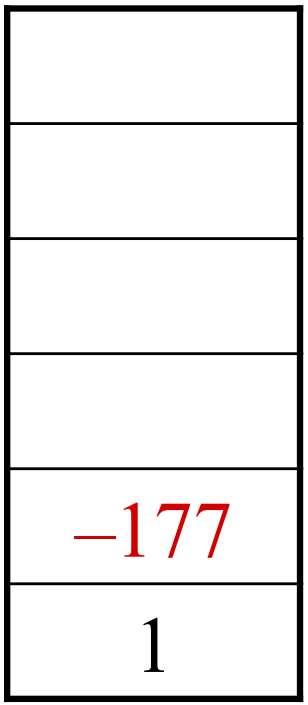
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Pop -182 and 5 and push $-182 + 5 = -177$

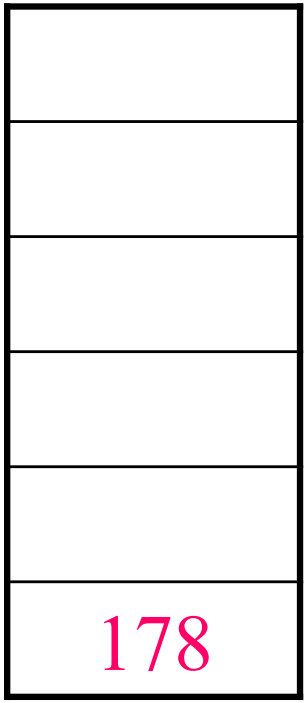
$$1\ 2\ 3\ +\ 4\ 5\ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ 9\ \times\ +$$



Reverse-Polish Notation

Pop -177 and 1 and push $1 - (-177) = 178$

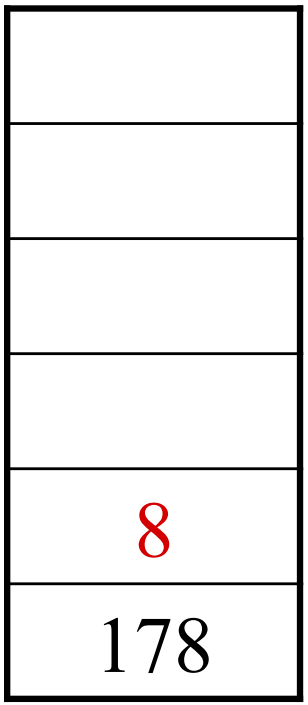
$$1\ 2\ 3\ +\ 4\ 5\ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ 9\ \times\ +$$



Reverse-Polish Notation

Push 8 onto the stack

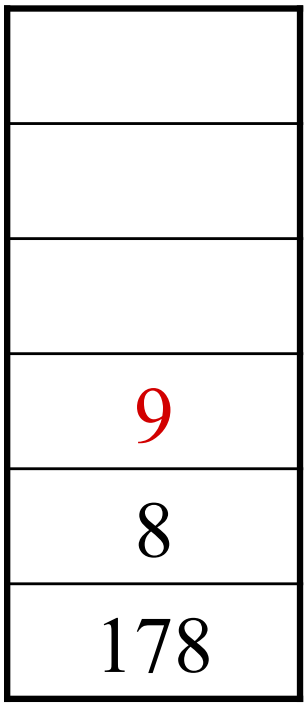
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Push 1 onto the stack

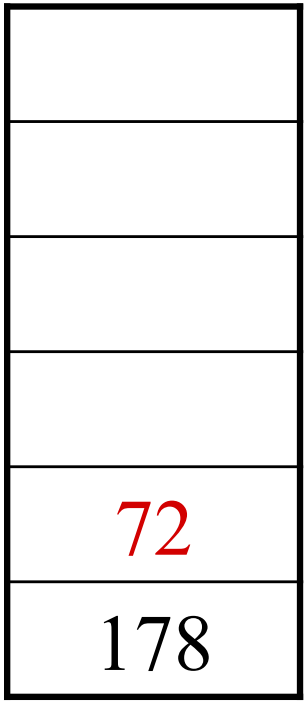
$$1\ 2\ 3\ +\ 4\ 5\ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ 9\ \times\ +$$



Reverse-Polish Notation

Pop 9 and 8 and push $8 \times 9 = 72$

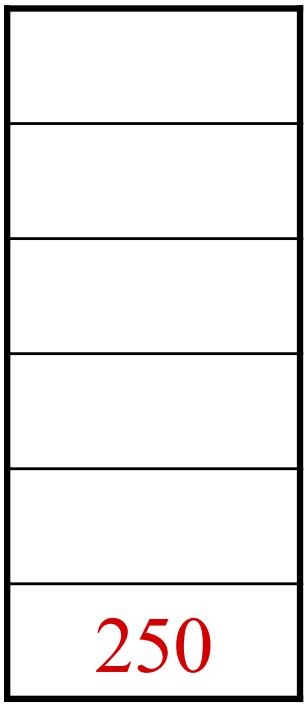
$$1\ 2\ 3\ +\ 4\ 5\ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ 9\ \times\ +$$



Reverse-Polish Notation

Pop 72 and 178 and push 178 + 72 = 250

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Thus

$$1\ \ 2\ \ 3\ +\ 4\ \ 5\ \ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ \ 9\ \times\ +$$

evaluates to the value on the top: 250

The equivalent in-fix notation is

$$((1 - ((2 + 3) + ((4 - (5 \times 6)) \times 7))) + (8 \times 9))$$

We reduce the parentheses using order-of-operations:

$$1 - (2 + 3 + (4 - 5 \times 6) \times 7) + 8 \times 9$$

Reverse-Polish Notation

Incidentally,

$$1 - 2 + 3 + 4 - 5 \times 6 \times 7 + 8 \times 9 = -132$$

which has the reverse-Polish notation of

$$1\ 2\ -\ 3\ +\ 4\ +\ 5\ 6\ 7\ \times\ \times\ -\ 8\ 9\ \times\ +$$

For comparison, the calculated expression was

$$1\ 2\ 3\ +\ 4\ 5\ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ 9\ \times\ +$$

Standard Template Library

The Standard Template Library (STL) has a *wrapper* class stack with the following declaration:

```
template <typename T>
class stack {
    public:
        stack();                    // not quite true...
        bool empty() const;
        int size() const;
        const T & top() const;
        void push( const T & );
        void pop();
};
```

Standard Template Library

```
#include <iostream>
#include <stack>
using namespace std;
int main() {
    stack<int> istack;

    istack.push( 13 );
    istack.push( 42 );
    cout << "Top: " << istack.top() << endl;
    istack.pop();                                  // no return value
    cout << "Top: " << istack.top() << endl;
    cout << "Size: " << istack.size() << endl;

    return 0;
}
```

Stacks

The stack is the simplest of all ADTs

- Understanding how a stack works is trivial

The application of a stack, however, is not in the implementation, but rather:

- where possible, create a design which allows the use of a stack

We looked at:

- parsing, function calls, and reverse Polish